Intermediate — an HTML + JavaScript frontend — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

- Hint 1 — two routes: a page and an API
- Hint 2 — the fetch call

Hint 1 — two routes: a page and an API

The backend serves two things:

- `GET /` → an HTML page (a box, a button, an empty result element).
- `POST /api/classify` → the JSON endpoint from the FastAPI exercise (`{"text": ...}` → `{"label": ...}`).

The page's JavaScript is what connects them — the browser calls your API, your API calls the model, and the key never leaves the server.

Hint 2 — the fetch call

The page's script reads the input, calls the API, and shows the result — no reload:

```
async function go() {
  const text = document.getElementById("text").value;
  const res = await fetch("/api/classify", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({text: text})
  });
  const data = await res.json();
  document.getElementById("out").innerText = "Result: " + data.label;
}
```

That `fetch` is exactly how a React or plain-JS frontend talks to your pipeline. Test the backend with `TestClient` (`GET /` returns the page, `POST /api/classify` returns the label); open the page in

a browser to see the fetch work live. HTMX (`hx-post`) is the no-JavaScript alternative. The full code is in the Solution.